Aeronautical and Astronautical Engineering Seaglider Capstone Continuity Document

Edward J Park

May 2026

Contents

1 Mission Overview

1.1 Mission Objective and Problem Statement

Autonomous underwater gliders are designed for long-duration ocean missions with minimal human intervention. However, their extended deployments and limited real-time communication capabilities make them susceptible to undetected faults. These faults can degrade vehicle performance, reduce mission efficiency, and in severe cases result in mission failure or loss of the vehicle.

A fault is defined as any physical or operational condition that causes the seaglider to deviate from its expected or commanded behavior. These deviations can impact key aspects of vehicle performance, including steering, buoyancy, stability, and energy consumption. In particular, faults may introduce asymmetries in control and hydrodynamic behavior, forcing the glider to make continuous corrective actions. These corrections increase battery usage and can ultimately prevent mission completion, even if navigation objectives are achieved.

The objective of this project is to develop a fault detection and identification tool that utilizes historical Seaglider datasets to detect and isolate wing or rudder loss within five datasets of occurrence, with a confidence level of 60%, while requiring no modifications to the Seaglider hardware or onboard control systems.

1.2 Purpose

The purpose of this continuity document is to provide future capstone teams with a centralized reference for the design, testing, documentation, and lessons learned throughout development of the Damage Assessment Protocol (D.A.P.) system. This document serves as a roadmap to the project's major deliverables and supporting documentation, allowing future teams to understand previous design decisions, reproduce testing activities, and continue development without having to reconstruct project knowledge from individual reports or source code.

This continuity document summarizes the architecture and functionality of the D.A.P. system while providing references to the primary design documents, test plans, test procedures, validation results, and supporting analyses generated during the project. Rather than replacing the original documents, this report directs readers to the appropriate resources needed to understand specific subsystems and project activities.

Future teams should use this document as a starting point when becoming familiar with the project. References throughout the document identify where detailed information can be found regarding the Digital Twin, Dynamics and Forces models, Diagnoser, Pilot Guidebook, subsystem testing, system testing, and future development recommendations.

The ultimate goal of this document is to preserve project knowledge and provide future teams with the information necessary to maintain, improve, and expand the D.A.P. system.

1.3 System Behavior and Error Characteristics

The Damage Assessment Protocol (D.A.P.) system identifies faults by comparing observed Seaglider behavior against behavior predicted by the Digital Twin. As a result, understanding how faults manifest within mission data is critical to understanding how the system operates.

Wing-loss and rudder-loss faults alter the hydrodynamic characteristics of the vehicle and produce measurable deviations in vehicle motion. These deviations are commonly observed in mission variables such as depth, pitch angle, roll angle, heading, vertical velocity, and control system activity. As fault severity increases, the difference between expected and observed vehicle behavior generally becomes more pronounced.

The D.A.P. system does not directly detect physical damage. Instead, it identifies abnormal behavior by comparing mission data against Digital Twin simulations. The Diagnoser evaluates percent differences between mission data and Digital Twin outputs and determines whether those

differences exceed predefined allowable margins. Sustained deviations above these allowable margins are interpreted as indicators of abnormal vehicle behavior.

Several sources of error can contribute to differences between mission data and Digital Twin outputs. These include environmental disturbances, sensor uncertainty, hydrodynamic coefficient uncertainty, vehicle parameter uncertainty, actuator variability, and modeling assumptions used within the Dynamics and Forces model. Consequently, some amount of difference between mission and simulated data is expected even during nominal operation.

Because the Diagnoser relies on Digital Twin outputs, the overall confidence of the D.A.P. system is directly dependent on Digital Twin accuracy. Improvements to vehicle modeling, hydrodynamic coefficients, environmental modeling, and parameter estimation will generally improve both Digital Twin performance and overall fault identification capability.

Future teams should recognize that reducing Digital Twin error remains one of the most effective methods for improving system confidence and overall D.A.P. performance.

1.4 System Overview

The Damage Assessment Protocol (D.A.P.) system was developed to identify wing-loss and rudder-loss faults in a deployed Seaglider using only historical mission data received during normal operations. The system operates without requiring modifications to the Seaglider hardware, onboard software, or mission execution process.

The D.A.P. system is composed of three primary subsystems that work together to assess vehicle health:

- **Digital Twin** – The Digital Twin recreates Seaglider behavior by using recorded mission data, vehicle parameters, and the Dynamics and Forces model to generate simulated dive profiles. Nominal and faulted Digital Twin simulations are used as reference cases for fault identification.
- **Diagnoser** – The Diagnoser compares mission data against Digital Twin outputs to determine whether abnormal behavior exists. If a fault is detected, the Diagnoser compares the mission data against multiple fault-case simulations and identifies the most likely fault condition.
- **Documentation Subsystem** – The Documentation subsystem consists of the Pilot Guidebook, Continuity Document, validation documentation, and supporting project records. These documents enable pilots to operate the D.A.P. system, understand system outputs, reproduce testing activities, and continue future development.

At a high level, the system begins with mission data collected by a deployed Seaglider. The Digital Twin uses this information to generate simulated vehicle behavior for nominal and faulted conditions. The Diagnoser then compares the mission data against the Digital Twin outputs and identifies the fault case that most closely matches the observed behavior. The results are provided to pilots through the D.A.P. outputs and supporting documentation.

Future teams should understand that the overall performance of the D.A.P. system is heavily dependent on the accuracy of the Digital Twin. Because the Diagnoser compares mission data against Digital Twin simulations, improvements to Digital Twin fidelity will generally produce the largest improvements in overall fault identification performance.

1.5 Operational Context and Constraints

The system is developed under the following constraints:

- No modification to onboard Seaglider hardware or embedded control software

- Limited real-time communication during deployment
- Dependence on available telemetry and sensor data

These constraints require the system to operate externally using post-processed or intermittently received data while still providing timely and actionable fault identification.

1.6 System Visualization and Architecture

To support understanding of the system and its operation, the following figures are included:

- Concept of Operations (CONOPS)
- System Architecture Diagram
- D.A.P. System Overview

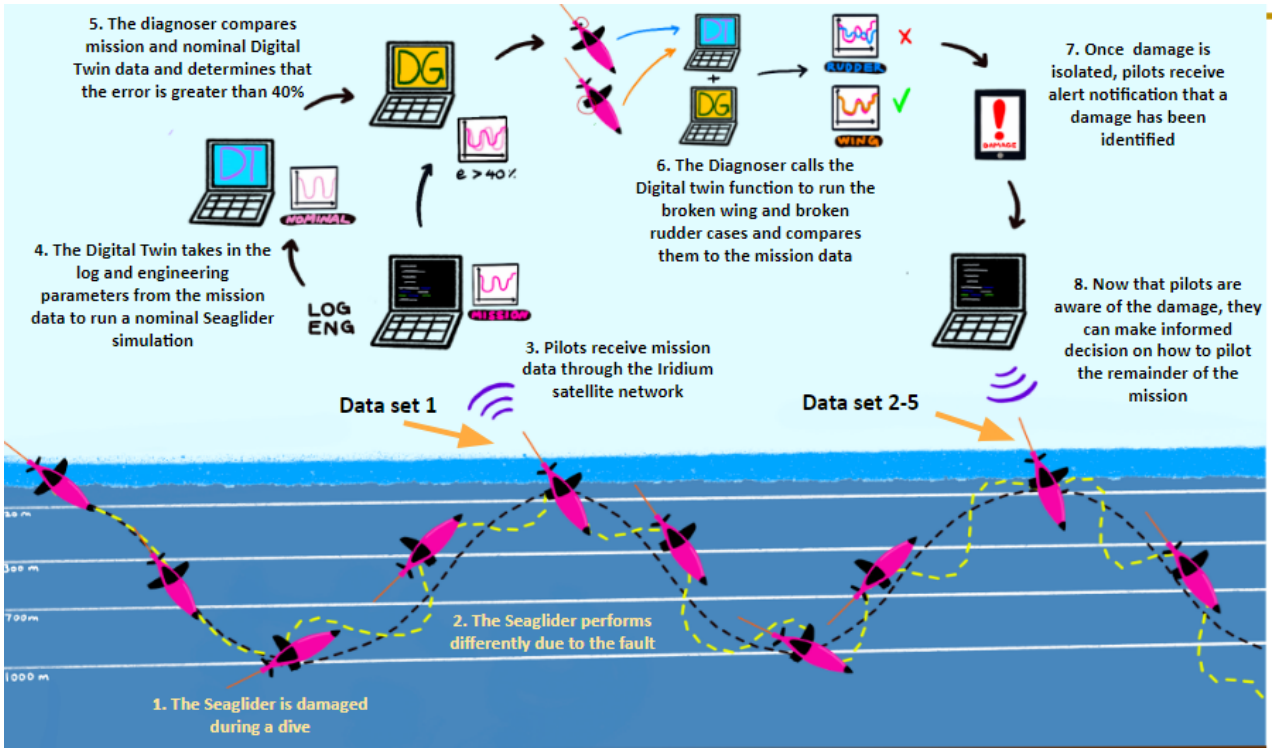

Figure 1: Concept of Operations (CONOPS) for the Seaglider Damage Assessment Protocol System

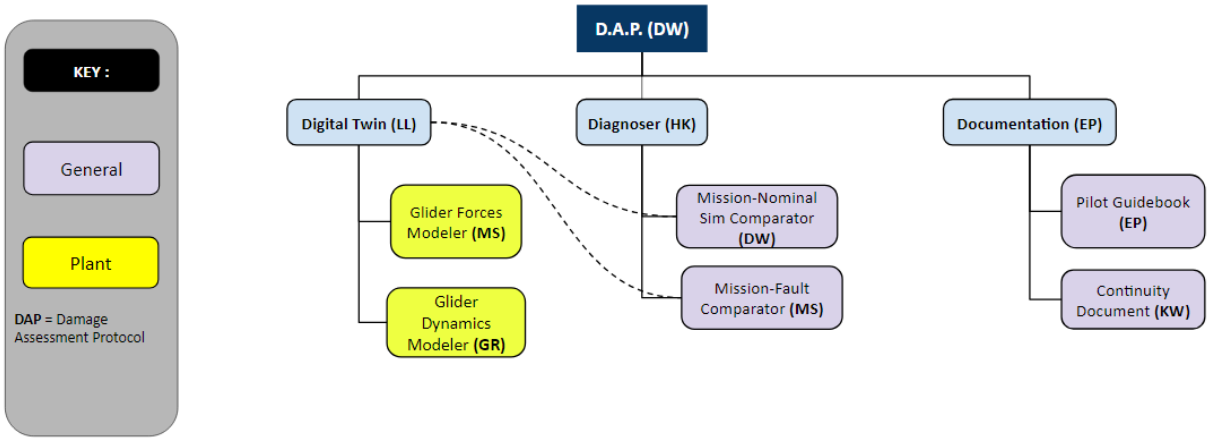

Figure 2: System Architecture of the D.A.P. System with Functional Systems: Digital Twin, Diagnoser, and Documentation

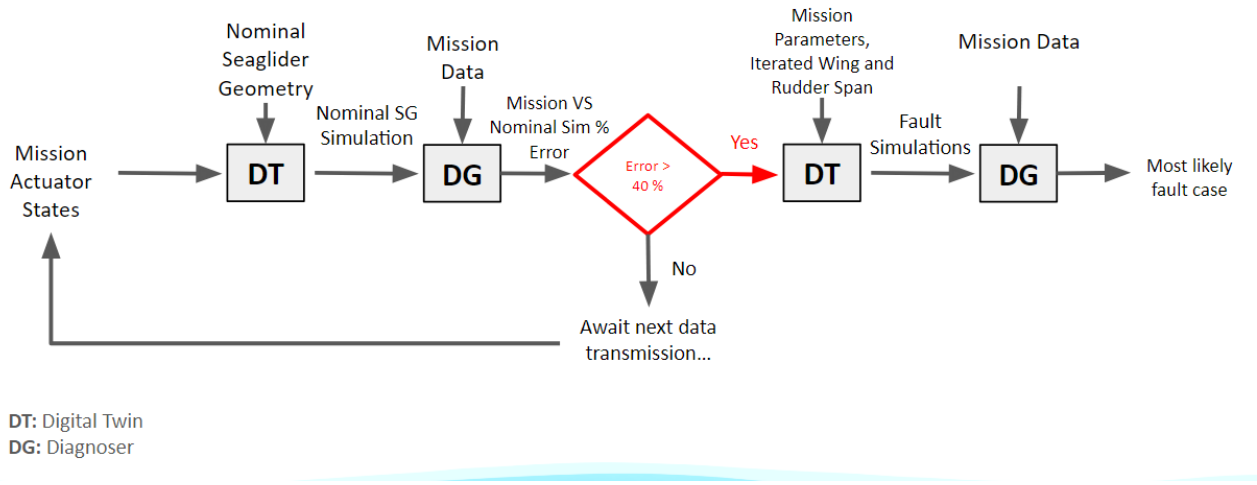

Figure 3: DAP System Overview and Data Flow

1.7 Continuity Document Overview

This document provides a complete record of the system design, implementation, and validation developed throughout the capstone project. It is intended to enable future teams to understand, reproduce, and extend the work completed.

2 Subsystem Requirements

The following subsection defines the requirements for each subsystem within the Damage Assessment Protocol (DAP) framework. Each subsystem contributes to the overall system functionality described in the system-level requirements and is responsible for specific roles in fault detection, diagnosis, and mitigation.

Subsystem requirements are organized by component to clearly define responsibilities and ensure traceability between subsystem behavior and overall system performance.

2.1 Digital Twin Requirements

The Digital Twin subsystem is responsible for generating expected Seaglider behavior under nominal conditions. It provides baseline predictions for vehicle states and forces, which are used for comparison against onboard-recorded data to support fault detection and diagnosis.

Digital Twin Functional Requirements

- **DTR-1:** The Digital Twin shall output orientation (pitch angle, roll angle, and heading) with percent error $\leq 19.80\%$ relative to onboard-recorded data.
- **DTR-2:** The Digital Twin shall output orientation rates (pitch angle rate, roll angle rate, and heading rate) with percent error $\leq 9.90\%$ relative to onboard-recorded data.
- **DTR-3:** The Digital Twin shall output vertical position with percent error $\leq 6.60\%$ relative to the onboard-recorded depth.
- **DTR-4:** The Digital Twin shall output vertical velocity with percent error $\leq 3.30\%$ relative to the onboard-recorded vertical velocity.
- **DTR-5:** The Digital Twin shall estimate net forces such as lift, drag, buoyancy $\leq 9.90\%$ error from the existing nominal planar flight hydrodynamic model.

Forces Modeler Requirements

- **FMR-1:** The Forces Modeler shall compute the lift force $\leq 3.30\%$ error of the lift force from the existing nominal planar flight hydrodynamic model.
- **FMR-2:** The Forces Modeler shall compute the drag force $\leq 3.30\%$ error of the drag force from the existing nominal planar flight hydrodynamic model.
- **FMR-3:** The Forces Modeler shall compute buoyant force $\leq 3.30\%$ error of buoyancy estimator on board of the Seaglider.

Dynamics Modeler Requirements

- **DNR-1:** Depth (z) of a dive must be modeled within 6.6% percent error of on board recorded depth of a dive.
- **DNR-2:** Vertical velocity (w) of a dive must be modeled within 3.3% percent error of on board recorded Vertical Speed dz/dt of a dive.
- **DNR-3:** Pitch of a dive must be modeled within 6.6% percent error of on board recorded pitch angle of a dive.
- **DNR-4:** Roll of a dive must be modeled within 6.6% percent error of on board recorded roll angle of a dive.

- **DNR-5:** Heading of a dive must be modeled within 6.6% percent error of on board recorded heading angle of a dive.
- **DNR-6:** Pitch rate of a dive must be modeled within 3.3% percent error of on board recorded pitch angle of a dive.
- **DNR-7:** Roll rate of a dive must be modeled within 3.3% percent error of on board recorded roll angle of a dive.
- **DNR-8:** Heading rate of a dive must be modeled within 3.3% percent error of on board recorded heading angle of a dive.

2.2 Diagnoser Requirements

The Diagnoser subsystem is responsible for identifying and classifying fault conditions by analyzing discrepancies between Digital Twin outputs and onboard-recorded mission data. It processes comparison results to determine the most likely fault case and its severity.

Diagnoser Functional Requirements

- **FDR-1:** The Diagnoser shall diagnose singular wing or rudder faults between 0-100% flight surface span remaining.

Fault Detector Requirements

- **MSR-1:** The Fault Detector shall trim the first and last 20% of each dive profile for Digital Twin and Mission datasets before performing any comparison.
- **MSR-2:** The Fault Detector shall compute the percentage difference between the trimmed Digital Twin and Mission datasets and shall output the percentage difference for variables exceeding 40% error.

Fault Isolator Requirements

- **FI-1:** The Mission Fault Comparator shall identify the lowest percent difference between single 0-100% wing or rudder DT cases and mission data.

2.3 Documentation Requirements

The Documentation subsystem is responsible for ensuring traceability, usability, and continuity of the DAP system. It captures system-level information, supports operator interaction through structured guidance, and enables future teams to reproduce and extend the work completed.

Documentation System Requirements

- **DOCR-1:** The documentation system shall ensure traceability between system requirements, design decisions, and verification test results for Digital Twin and the Diagnoser.
- **DOCR-2:** The documentation system shall provide an instruction manual with sufficient DAP system output interpretation, troubleshooting methods, and usage examples such that the DAP system can be used by the pilots without further input.

Pilot Guidebook Requirements

- **PG-1:** The Pilot Guidebook shall provide procedures for pilots to use the DAP system to help identify the corresponding fault case and interpret all of DAP system outputs.
- **PG-2:** The pilot guidebook shall enable pilots to understand the calculations and the origin of all system outputs produced by the DAP system for every pilot command input.

Capstone Continuity Document Requirements

- **CCD-1:** The Continuity Document shall store sufficient test configuration information to allow all system tests to be reproduced by a subsequent project team.
- **CCD-2:** The Continuity Document shall record test procedures for all executed system tests, including test conditions, input parameters, outputs, and pass/fail outcomes, to support verification of the Diagnoser and Digital Twin
- **CCD-3:** The Continuity Document shall record the results of Digital Twin validation tests conducted in the field deployment tests and wind tunnel experiments.
- **CCD-4:** The Continuity Document shall record the results of Diagnoser validation tests conducted during at least 1 field deployment test.

3 System Design

This section provides an overview of the design and implementation of each major subsystem developed as part of the Damage Assessment Protocol (D.A.P.) system. The purpose of this section is not to reproduce every design document, but rather to guide future teams through the system architecture, explain the role of each subsystem, and identify the supporting documentation that contains detailed design information.

For each subsystem, this section summarizes its purpose, functionality, major design decisions, implementation approach, and relationship to the overall D.A.P. architecture. References to supporting documentation are provided throughout to assist future teams in location detailed equations, code implementations, assumptions, and other analyses.

Future teams should use this section as a roadmap to understand how the D.A.P. system was constructed and how the individual subsystems interact. Before making modifications to any subsystem, teams are encouraged to review both the summary provided here and the associated supporting documentation references with each subsection.

The following subsystems are discussed in this section:

- Digital Twin including Dynamics and Forces
- Diagnoser
- Mitigator
- Documentation

3.1 Digital Twin

3.1.1 Overview

The Digital Twin (DT) is the primary simulation component of the Damage Assessment Protocol (D.A.P.) system. Its purpose is to recreate the behavior of a deployed Seaglider using recorded

mission data, vehicle parameters, environmental conditions, and the Dynamics and Forces model. The Digital Twin generates simulated vehicle states that can be directly compared against mission data to identify abnormal vehicle behavior and support fault diagnosis.

The Digital Twin serves as the foundation of the Diagnoser subsystem. Any inaccuracies within the Digital Twin will directly affect the fault identification capability of the D.A.P. system. As a result, maintaining and validating the Digital Twin should be considered one of the highest priorities for future development teams.

3.1.2 Supporting Documentation

The complete design, implementation, and evolution of the Digital Twin is documented within the Digital Twin Design Documentation.

This document should be considered the primary reference for understanding how mission data is converted into a simulated Seaglider dive and how Digital Twin outputs are generated.

3.1.3 How to Use the Digital Twin Design Documentation

Future teams attempting to understand the overall architecture of the Digital Twin should begin with Sections 2.1 through 2.3. These sections explain the Mission Data Matrix, Digital Twin architecture, required inputs, simulation workflow, and overall execution sequence used throughout the subsystem.

Section 2.1 describes how mission data is extracted from Seaglider mission files and converted into a Mission Data Matrix. This section should be referenced whenever modifications are made to data extraction, preprocessing routines, or mission file formats.

Section 2.2 provides an overview of the Digital Twin architecture and describes the major inputs required for simulation, including mission data, parameter files, and hydrodynamic coefficient files. Future teams modifying subsystem interfaces or simulation inputs should begin with this section.

Section 2.3 explains how the Digital Twin executes throughout a dive. This section describes the simulation loop, state updates, actuator inputs, and generation of the Nominal Simulation Matrix. Teams modifying simulation execution logic should review this section before making changes.

3.1.4 Mission Data Matrix Generation

The Mission Data Matrix serves as the primary interface between real Seaglider mission data and the Digital Twin. Mission data extracted from Seaglider files is organized into a structured matrix containing vehicle states, actuator positions, environmental information, and mission parameters required by the simulation.

Future teams modifying mission data extraction or introducing additional measured variables should review Section 2.1 and the associated code implementation before making changes. Since all Digital Twin simulations depend on the Mission Data Matrix, errors introduced at this stage will propagate throughout the entire D.A.P. system.

3.1.5 Simulation Architecture

The final Digital Twin architecture utilizes recorded actuator positions directly from Seaglider mission files. Battery position, battery roll angle, and Variable Buoyancy Device (VBD) commands are extracted from mission data and used as simulation inputs throughout the dive.

This design decision was made after earlier architectures attempted to independently estimate actuator behavior and vehicle control decisions. Due to limitations associated with reproducing the proprietary Seaglider controller, the project team transitioned to a mission-data-driven approach. This significantly improved simulation fidelity while reducing uncertainty associated with control estimation.

Future teams interested in developing predictive or real-time Digital Twin capabilities should review Section 5 of the Digital Twin Design Documentation, which describes the original architecture and design evolution.

3.1.6 Code Structure and Implementation

The detailed Digital Twin code walkthrough is provided within Section 4 of the Digital Twin Design Documentation.

Future teams attempting to modify simulation behavior should review:

- Section 4.5 – Simulation initialization.
- Section 4.6 – Parameter loading and updates.
- Section 4.7 – Mission data processing.
- Section 4.8 – Control vector generation.
- Section 4.9 – State vector generation.
- Section 4.10 – Dynamics and Forces model execution.
- Section 4.11 – ODE integration.
- Section 4.12 – State propagation.
- Section 4.13 – Output generation.
- Section 4.14 – Nominal Simulation Matrix creation.
- Section 4.15 – Data export and storage.

These sections collectively describe how mission inputs are transformed into Digital Twin outputs and should be consulted before any modifications are made to simulation logic.

3.1.7 Design Changes

The original Digital Twin concept differed significantly from the final implementation. Earlier designs included independent actuator models, energy consumption estimation, waypoint navigation logic, and control decision modeling intended to replicate Seaglider behavior without relying on completed mission data.

As development progressed, it became clear that accurately reproducing the proprietary Seaglider control algorithms would introduce substantial uncertainty into the simulation. The project team therefore adopted the current architecture, which directly utilizes recorded actuator states from mission files.

This decision improved simulation accuracy and validation performance but introduced a key limitation: the Digital Twin can only operate after mission data has been received. Consequently, the current implementation operates one dive behind the vehicle rather than providing real-time fault detection.

Future teams exploring predictive Digital Twin architectures should review the historical designs documented within Section 5 before attempting implementation.

3.1.8 Recommendations for Future Teams

Future teams should revalidate the Digital Twin whenever modifications are made to:

- Dynamics and Forces models.
- Hydrodynamic coefficients.
- Vehicle mass properties.
- Inertia matrices.
- Mission data processing routines.
- Fault-case definitions.
- State vector definitions.

Because Diagnoser performance is directly dependent on Digital Twin accuracy, modifications should be verified against mission data before integration into the D.A.P. system. Improving Digital Twin fidelity remains one of the most impactful areas for future development and offers the greatest opportunity for improving overall fault detection performance.

3.1.9 Dynamics and Forces

The Dynamics and Forces model serves as the simulation foundation of the Digital Twin subsystem. It is responsible for modeling Seaglider motion by calculating hydrodynamic forces, buoyancy forces, moments, vehicle kinematics, and state propagation. The resulting simulated vehicle states are used by the Digital Twin to generate expected dive behavior for comparison against real mission data.

The Dynamics and Forces development effort is documented across six supporting documents located within the Digital Twin/Dynamics folder. Future teams should use these documents collectively when modifying or validating the model.

3.1.10 Dynamics and Forces Documentation

The Dynamics and Forces Documentation is the primary design document for the Dynamics and Forces model. Future teams should begin with this document when attempting to understand how the simulation operates.

The document introduces the six-degree-of-freedom nonlinear dynamics architecture used throughout the Digital Twin and explains how the vehicle state is propagated from one timestep to the next. It contains the complete derivation of the equations used to model vehicle motion and force generation.

Future teams should reference:

- Section II.A for modeling assumptions.
- Section II.B for coordinate system definitions.
- Section II.C for state and control vector definitions.
- Section II.D for coordinate transformations.
- Section II.E for flow angle calculations.
- Section II.F for hydrodynamic force calculations.
- Section II.G for buoyancy and gravity modeling.

- Section II.H through II.K for mass, inertia, battery motion, center-of-gravity offsets, and moment calculations.
- Section II.L through II.N for vehicle kinematics, equations of motion, and state propagation.
- Sections III and IV for subsystem requirements and error budget allocations.

Any modifications to the physics model should be documented within this report and validated against the existing requirements.

3.1.11 SG194 Mass and Inertia Matrix Calculation

This document contains the methodology used to calculate all mass and inertia properties required by the Dynamics and Forces model.

Future teams should use this document whenever modifications are made to:

- Vehicle geometry.
- Vehicle mass properties.
- Wing configurations.
- Rudder configurations.
- Fault-case definitions.
- Center-of-gravity assumptions.

The document begins with vehicle parameters extracted from the SG194 trim and balance sheet and proceeds through the calculation of battery mass properties, hull geometry assumptions, added mass matrices, added inertia matrices, and stationary inertia matrices.

The fault-case section provides the mass and inertia matrices used for the nominal, 50

Particular attention should be given to the Vehicle CG Shift Analysis section, which justifies the assumption that fault-induced center-of-gravity shifts are negligible for the currently implemented fault cases.

3.1.12 Dynamics Test Plan

The Dynamics Test Plan defines how the Dynamics model was verified and validated against real Seaglider mission data.

The purpose of the testing was to determine whether the Dynamics model could accurately reproduce measured vehicle behavior throughout a dive. The plan establishes the Dynamics Requirements (DNR-1 through DNR-8) and defines allowable error margins for depth, vertical velocity, pitch, roll, heading, and angular rate predictions.

Future teams should review this document before modifying the Dynamics model because any significant model change should be accompanied by revalidation using the same methodology and acceptance criteria. This document also identifies the parameters, variables, measurements, and mission data used throughout validation testing.

3.1.13 Dynamics Test Procedure

The Dynamics Test Procedure provides the exact step-by-step process used to execute Dynamics model validation testing.

Future teams performing regression testing should follow this procedure to maintain consistency with previous validation efforts.

The procedure includes:

- Simulation setup.
- Software initialization.
- Initial state vector generation.
- Control vector generation.
- Dynamics integration.
- Post-processing.
- Requirement verification.
- Data collection and documentation.

This document should be used whenever modifications are made to the Dynamics model to ensure that validation results remain comparable to previous testing campaigns.

3.1.14 Forces Test Procedure

The Forces Test Procedure focuses specifically on validating the force calculations used by the Dynamics and Forces model.

Unlike the Dynamics Test Procedure, which evaluates overall vehicle behavior, this procedure evaluates the individual lift, drag, and buoyancy calculations generated by the Forces model.

The procedure uses CFD-derived aerodynamic and hydrodynamic coefficient data to verify that the implemented force equations correctly reproduce expected physical behavior.

Future teams modifying:

- Lift coefficient tables.
- Drag coefficient tables.
- Buoyancy calculations.
- CFD datasets.
- Force-generation equations.

should repeat the validation activities described within this document before integrating changes into the Digital Twin.

3.1.15 Dynamics and Forces Test Results

The Dynamics and Forces Test Results document contains the historical validation record of the subsystem.

The document records both pre-TRR and post-TRR testing efforts and documents how the model evolved throughout development. Future teams should review this document before making modifications to understand previously encountered issues, implemented fixes, and remaining limitations.

Particularly useful sections include:

- Pre-TRR Dynamics Results.
- Post-TRR Dynamics Results.
- Forces Validation Results.

- Additional Dynamics Verification Tests.

The results document provides insight into which model changes improved performance and which areas remain candidates for future improvement. It should be treated as the primary historical record of Dynamics and Forces subsystem validation.

3.1.16 Recommendations for Future Teams

Future teams should review the documents in the following order:

1. Dynamics and Forces Documentation.
2. SG194 Mass and Inertia Matrix Calculation.
3. Dynamics Test Plan.
4. Dynamics Test Procedure.
5. Forces Test Procedure.
6. Dynamics and Forces Test Results.

The first two documents explain how the model was developed, while the remaining four documents explain how the model was tested and validated. Any modifications to the Dynamics and Forces model should be accompanied by updates to the associated documentation and revalidation using the established test procedures.

3.2 Diagnoser

3.2.1 System Overview

The Diagnoser is a diagnostic engine designed to identify, classify, and isolate faults in a seaglider. It achieves a successful diagnosis by comparing deviations between nominal and mission datasets. The Diagnoser is divided into two primary portions, that being, anomaly detection and fault isolation.

3.2.2 Code and Implementation

Anomaly Detection is handled by the `missioncompare_detector.m` function. At first, the function extracts times and variable names, then trims them. The detector begins by extracting variable names from the first row of each input matrix and pulling the corresponding time vectors:

```
varNames_DT = string(DT_Output(1, :));
varNames_MI = string(MI_Output(1, :));

time_DT = cell2mat(DT_Output(2:end, 1));
time_MI = cell2mat(MI_Output(2:end, 1));
```

These time vectors are required before normalization because the DT and MI outputs may differ in length.

Time Normalization:
To ensure a valid comparison, the detector trims both datasets to the same number of samples:

```
len_DT = length(time_DT);
len_MI = length(time_MI);
minLen = min(len_DT, len_MI);

time_DT = time_DT(1:minLen);
time_MI = time_MI(1:minLen);

DT_Output = DT_Output([1, 2:minLen+1], :);
MI_Output = MI_Output([1, 2:minLen+1], :);
```

This guarantees that every DT sample aligns with a corresponding MI sample.

Consistency Checks:
Before continuing, the detector verifies that:

- both datasets contain the same number of variables,
- variable names match exactly, and
- time vectors have equal length.

If any mismatch occurs, the function aborts with a descriptive error:

```
if size(DT_Output, 2) ~= size(MI_Output, 2)
    error('MissionCompare:VariableCountMismatch', ...
          'Digital Twin and Mission data have different numbers of variables.');
end

if ~isequal(varNames_DT, varNames_MI)
    error('MissionCompare:VariableNameMismatch', ...
          'Variable names do not match between DT and MI data.');
end
```

These checks ensure that the comparison is valid and that the DT and MI datasets are structurally aligned.

Struct Construction:
After validation, the detector converts each variable column into a field of a struct, enabling clean variable-by-variable access:

```
DT = struct();
MI = struct();

for i = 1:size(DT_Output, 2)
    varNameChar = char(varNames_DT(i));
    DT.(varNameChar) = cell2mat(DT_Output(2:end, i));
    MI.(varNameChar) = cell2mat(MI_Output(2:end, i));
end
```

This struct-based format is required for the later percent-difference computation and windowed binary detection.

Diagnose Struct Initialization:
The detector initializes a `Diagnose` struct to store all computed metrics, flags, and outputs for each variable. This struct provides a unified container for percent differences, window scores, excursions, and plot paths.

```
Diagnose = struct();
Diagnose.VariableNames = varNames_DT;  % store variable names
Diagnose.Diff          = struct();     % final Diff metric per variable
Diagnose.Failures      = struct();     % variables that fail threshold
Diagnose.Plots         = struct();     % file paths to saved plots
Diagnose.Excursions    = struct();     % time intervals of excursions
Diagnose.Count         = struct();     % count of samples above tolerance
Diagnose.Excess        = struct();     % total excess difference (sum over mission)
Diagnose.WindowScores  = struct();     % per-window scores (vector per variable)
Diagnose.RemovedZeros  = struct();     % number of zero-value samples removed
Diagnose.difference    = struct();     % percentage difference for comparison
```

Tolerance Definitions:

Each variable is assigned a percent-difference tolerance. Certain attitude variables use higher tolerances, while all others fall back to a default value:

- Pitch, Roll, Heading: 7.5–187.5%
- Depth: 14.5%
- All other variables: default tolerance of 2%

These tolerances are stored in a `containers.Map` for fast lookup:

```
toleranceMap = containers.Map( ...
    {'eng_pitchAng','eng_rollAng','eng_head','depth'}, ...
    [7.5,    187.5,   13.5,         14.5] ...
);

defaultTolerance = 2.0;
```

Windowing Parameters:

The binary-window detector uses fixed-size windows and a failure threshold:

```
windowSize       = 20;   % samples per window
failureThreshold = 40;   % window fails if >40% of samples exceed tolerance
```

A variable is marked as failed only if a flagged window occurs and it is either the final window or all subsequent windows are also flagged.

Plot Directory Setup:

A folder is created to store diagnostic plots generated during processing:

```
saveFolder = fullfile(pwd, 'MS_Comparator_Plots');
if ~exist(saveFolder, 'dir')
    mkdir(saveFolder);
end
```

Main Variable Loop:

The detector iterates through each variable (skipping the time column) and performs cleaning, trimming, tolerance assignment, and percent-difference computation.

```
for i = 2:length(varNames_DT)
    varName     = varNames_DT(i);
    varNameChar = char(varName);

    DT_raw = DT.(varNameChar);
    MI_raw = MI.(varNameChar);
```

Zero-Value Removal:

Samples where either DT or MI is zero are removed to avoid division-by-zero and eliminate invalid data:

```
validMask = (DT_raw ~= 0) & (MI_raw ~= 0);
numRemoved = sum(~validMask);
Diagnose.RemovedZeros.(varNameChar) = numRemoved;

DT_clean = DT_raw(validMask);
MI_clean = MI_raw(validMask);
t_clean  = time(validMask);
```

If no valid samples remain, the variable is skipped and placeholders are stored.

Middle 60% Trimming:

To avoid startup and end-of-dive transients, only the middle 60% of the mission is used:

```
trimMask = (t_clean > t0 + 0.2*T) & (t_clean < t0 + 0.8*T);

DT_trim = DT_clean(trimMask);
MI_trim = MI_clean(trimMask);
t_trim  = t_clean(trimMask);
```

Variables with no remaining samples after trimming are skipped.

NaN Removal:

Any NaN values are removed from the trimmed arrays:

```
nanMask = isnan(DT_trim) | isnan(MI_trim);

DT_trim(nanMask) = [];
MI_trim(nanMask) = [];
t_trim(nanMask)  = [];
```

Tolerance Assignment:

Each variable receives its tolerance from the map or the default value:

```
if isKey(toleranceMap, varNameChar)
    tolerance = toleranceMap(varNameChar);
else
    tolerance = defaultTolerance;
end
```

Percent-Difference Computation:

For each variable, the detector computes the percent difference between Mission (MI) and Digital Twin (DT) values using DT as the reference:

```
E = abs((MI_trim - DT_trim) ./ DT_trim) * 100;
```

This array `E` is stored for later comparison and plotting:

```
Diagnose.difference.(varNameChar) = E;
```

Excursion Detection:
Samples exceeding the tolerance are identified using a logical mask:

```
excursionMask = E > tolerance;
Diagnose.Count.(varNameChar) = sum(excursionMask);
```

The total excess difference above the tolerance is computed as:

```
excessdifference = max(E - tolerance, 0);
Diagnose.Excess.(varNameChar) = sum(excessdifference);
```

Binary Windowed Detector
The detector converts the percent-difference array into a binary error mask:

```
errorMask = E > tolerance;
```

The mission is divided into fixed-size windows:

```
numSamples = length(errorMask);
numWindows = ceil(numSamples / windowSize);

windowErrorRates = zeros(numWindows,1);
windowFlags      = false(numWindows,1);
```

Each window is evaluated independently:

```
for w = 1:numWindows
    idxStart = (w-1)*windowSize + 1;
    idxEnd   = min(w*windowSize, numSamples);

    windowErrors = errorMask(idxStart:idxEnd);

    windowErrorRates(w) = (sum(windowErrors) / length(windowErrors)) * 100;
    windowFlags(w) = windowErrorRates(w) > failureThreshold;
end
```

Window scores and failure indices are stored:

```
Diagnose.WindowScores.(varNameChar) = windowErrorRates;
Diagnose.WindowFailCount.(varNameChar) = sum(windowFlags);
Diagnose.TotalWindows.(varNameChar) = numWindows;
Diagnose.FailedWindowIndices.(varNameChar) = find(windowFlags);
```

Run-to-End Failure Logic:
A variable is marked as failed only if a flagged window occurs and it is either the final window or all subsequent windows are also flagged:

```
fail = false;

for w = 1:numWindows
    if windowFlags(w)
        if w == numWindows || all(windowFlags(w:end))
            fail = true;
            break;
        end
    end
end
```

The maximum window error rate is stored as the variable's Diff score:

```
Diagnose.Diff.(varNameChar) = max(windowErrorRates);

if fail
    Diagnose.Failures.(varNameChar) = max(windowErrorRates);
end
```

Excursion Interval Extraction:
Continuous segments of excursion samples are grouped into time intervals:

```
idx = find(excursionMask);
intervals = [];

if ~isempty(idx)
    d      = diff(idx);
    breaks = [0; find(d > 1); length(idx)];

    for b = 1:length(breaks)-1
        seg = idx(breaks(b)+1 : breaks(b+1));
        intervals = [intervals; t_trim(seg(1)) t_trim(seg(end))];
    end
end

Diagnose.Excursions.(varNameChar) = intervals;
```

Plot Generation
For each variable, a DT vs. MI plot is generated with tolerance bands:

```
fig = figure('Visible','off');

t_plot  = t_trim(:);
DT_plot = DT_trim(:);
MI_plot = MI_trim(:);

plot(t_plot, DT_plot, 'b-s', 'LineWidth', 1.5);
hold on;
plot(t_plot, MI_plot, 'r-o', 'LineWidth', 1.5);

lower = DT_plot * (1 - tolerance/100);
upper = DT_plot * (1 + tolerance/100);
```

```
plot(t_plot, lower, 'k--', 'LineWidth', 1);
plot(t_plot, upper, 'k--', 'LineWidth', 1);

xlabel('Time (minutes)');
ylabel(varNameChar);
legend('DT','MI', ...
       sprintf('-%g%%',tolerance), sprintf('+%g%%',tolerance), ...
       'Location','best');

title(['DT vs MI: ' varNameChar]);
grid on;

filePath = fullfile(saveFolder, varNameChar + ".png");
saveas(fig, filePath);
close(fig);

Diagnose.Plots.(varNameChar) = filePath;
```

Summary Output:
At the end of execution, a formatted summary is printed to the command window:

```
fprintf('\n============================================================\n');
fprintf('MISSION COMPARATOR SUMMARY\n');
fprintf('Window-based failure threshold: %.2f\n', failureThreshold);
fprintf('============================================================\n');
```

Each variable is listed with its PASS/FAIL status, Diff score, and failed window indices.

Fault Isolation is handled by the `D_fault_isolator` function. As an input the fault isolator can take in any number of cases via the `varargin` input. The function generate an array of zeroes to match the number of cases provided. Case names are also pulled for later use.

```
function [RankedTable, BestMatch] = D_fault_isolator(varargin)
    numLibraryCases = nargin;
    finalScores = zeros(numLibraryCases, 1);
    caseNames = cell(numLibraryCases, 1);
```

After generating the zeros and case names the isolator iterates through the zero array and fills it in. If the case names are empty it will generate a case name, otherwise, it will use the pulled case name.

```
    for i = 1:numLibraryCases
        varName = inputname(i);
        if isempty(varName)
            caseNames{i} = sprintf('Case_%d', i);
        else
            caseNames{i} = varName;
        end
        Resultcur = varargin{i};
        compStruct = Resultcur.difference;
        vars = fieldnames(compStruct);
```

For each case a mean error is calculated and the error sum is updated. This is fed into `finalScores(i)`, which will provide the ranking metric later.

```
        error_sum = 0;
        count = 0;
        for v = 1:numel(vars)
            arr = compStruct.(vars{v});
            mean_error = mean(arr,'omitnan');
            if ~isnan(mean_error)
                error_sum = error_sum + mean_error;
            end
        end
        finalScores(i) = error_sum / count;

    end
```

Finally a table is created that sorts cases by their final score in ascending order. `BestMatch` pulls the highest score to provide the main output of the fault isolator.

```
RankedTable = table(caseNames,finalScores,'VariableNames', ...
{'Digital_Twin_Library_Case', 'Percent_Match_Error'});
RankedTable = sortrows(RankedTable, 'Percent_Match_Error', 'ascend');
BestMatch = RankedTable.Digital_Twin_Library_Case{1};
```

3.2.3 Fault Isolation

Once the Detector identifies a sustained failure, the Diagnoser system proceeds to execute the Fault Isolator. The Fault Isolator produces a table of possible faults and sorts them by their confidence metric. The final diagnosed fault is the one with the lowest confidence metric.

- **Fault Classification:** The system operates by having generated damage modes. These are sorted into by percent loss of rudder or wing.
- **Fault Matching/Confidence Metric:** The Fault Isolator compares mission data against each of the generated damage modes. First it calculates the mean percentage error across all the variables. The simulated cases are sorted based on average percent error. The simulation that shows the lowest error is classified as the **Best Match**. The average percent error works as an inverse confidence metric. The lower the percent error the higher degree of certainty that the seaglider is experiencing the recomended fault

3.3 Mitigator

The Mitigator was a preliminary design of the previous D.A.M.P. design to allow the Seaglider pilots to simulate different dive profiles to determine the most efficient pilot commands needed to conserve the most energy during a mission if damage to the Seaglider was detected.

Utilizing the previous Digital Twin design, which included the battery and buoyancy systems to model the different dive profiles that would occur. This would result in inputting the Dive Parameters, Seaglider Characteristics, and Environmental Parameters for the mission to enable the Mitigator to run the Digital Twin with an iterative dive profile generator. The Mitigator would then filter out the mission drive datasets to output dive profiles which had the minimum battery usage with the longest distance achieved for the pilots. Enabling the pilots to place input pilot commands for the best dive profile for a damaged Seaglider, in order to mitigate the battery loss from excessive correction from the Seaglider.

To state again, this is only a preliminary design that can be subject to change and only an idea of what could have been for the Mitigator. Given the changes to the current D.A.P. design, this preliminary model of the Mitigator cannot be utilized. As such, this serves as an idea of how the Mitigator would be designed and various applications that needed to be known in order for the Mitigator to operate functionally with the Digital Twin and Diagnoser.

3.3.1 Iterative Dive Profile Generator

The Iterative Dive Profile Generator is the first process of the Mitigation, as it utilizes the Digital Twin to generate different dive profiles by changing pilot commands, such as D_TGT, T_DIVE, T_MISSION. The inputs were still to be determined at the time the project was downscoped however. With the operations of the Digital Twin as the iterative dive profile generator, it would have the limits set by the Seaglider characteristics of pitch, roll, and VBD in A/D counts. The Digital Twin would generate up to 1000 iterations of simulations of the given mission parameters, Seaglider characteristics, environment parameters, and targets file until one of the required operation limits was reached. Once an operation limit is reached, the generator will end the dive profile generation and will enable the Mitigation Filter.

3.3.2 Mitigation Filter

The Mitigation Filter would then serve as filtering the dive profiles generated from the Iterative Dive Profile Generator within the mission constraints. Where the pilot will be given an option to adjust the tolerance of the mission constraints and operational impact for the Mitigator. This would filter dive profiles to the limits of the depth, distance of the Seaglider, and velocity ranges of the Seaglider depending on the tolerance that the pilot wants to select. Which would result in the desired dive profile for the minimum battery case and max distance achieved given. The output would be a dive profile similar to seaglider.pub, which would show a simulated dive profile plot and the amount of energy consumed from performing the maneuver.

3.4 Documentation

The Documentation subsystem was developed to support system usability, engineering traceability, and long-term project continuity for the Damage Assessment Protocol (D.A.P.) system. Unlike the Digital Twin and Diagnoser subsystems, which focus on simulation and fault identification, the Documentation subsystem exists to ensure that future operators and development teams can effectively use, understand, verify, and continue development of the D.A.P. system.

The Documentation subsystem consists of two primary deliverables:

- Pilot Guidebook
- Capstone Continuity Document

Together, these documents satisfy the Documentation subsystem requirements by providing operational guidance, output interpretation, design traceability, test documentation, and subsystem continuity.

3.4.1 Pilot Guidebook

The Pilot Guidebook serves as the operator-facing document of the D.A.P. system. Its purpose is to enable pilots and operators to independently execute the D.A.P. system, understand its outputs, and relate the resulting fault diagnosis to observed Seaglider behavior. The guidebook was designed to satisfy requirements PG-1 and PG-2 by providing both operational procedures and explanations of how D.A.P. results are generated.

Future teams updating the D.A.P. software should ensure that corresponding updates are reflected within the Pilot Guidebook. The current guidebook is organized into four major sections.

Section 1: Pilot Guidebook Overview This section introduces the D.A.P. system, explains its purpose, identifies the currently supported fault cases, and describes how the guidebook should be used. Future teams should update this section whenever additional fault cases, capabilities, or operational limitations are added to the system.

Section 2: D.A.P. Manual This section provides step-by-step instructions for operating the D.A.P. system in MATLAB. The procedure guides the user through opening the D.A.P. folder, launching the DAP.Main script, importing the required .log and .nc mission files, and executing the system to generate a Final Damage Assessment Report. Future teams should update this section whenever the software workflow, required inputs, execution environment, or user interface changes.

Section 3: D.A.P. Output and Interpretation This section explains how to interpret the Final Damage Assessment Report produced by the D.A.P. system. It defines the Primary Diagnosis, Average Percent Error, and fault ranking methodology used by the Diagnoser. Future teams should update this section whenever new confidence metrics, fault-ranking methods, or output formats are introduced.

Section 4: Understanding D.A.P. Results This section provides a detailed explanation of how the D.A.P. system generates a diagnosis. Topics include Mission Data Matrix generation, Digital Twin comparison, allowable tolerances, percent difference calculations, window-based failure detection, sustained failure logic, fault isolation, and verification using seaglider.pub. This section should be considered the primary reference for operators seeking to understand how their mission inputs ultimately produce a fault diagnosis. Future teams modifying comparison thresholds, fault detection algorithms, Mission Data Matrix variables, or Diagnoser logic should ensure this section remains synchronized with the software implementation.

3.4.2 Capstone Continuity Document

The Continuity Document serves as the engineering handoff document for future development teams. Its purpose is to preserve subsystem designs, supporting documentation, implementation methods, validation activities, test results, design decisions, limitations, and recommendations developed throughout the capstone project.

Future teams should use the Continuity Document as the starting point for understanding the overall D.A.P. architecture before reviewing the subsystem-specific design documents. The document is intended to satisfy continuity requirements CCD-1 through CCD-4 by providing sufficient information to reproduce testing, understand subsystem implementations, and continue future development efforts.

3.4.3 Recommendations for Future Teams

Documentation should be treated as a system component rather than a project deliverable. Whenever modifications are made to the Digital Twin, Diagnoser, fault definitions, Mission Data Matrix variables, comparison logic, output formats, or operational procedures, the Pilot Guidebook and Continuity Document should be updated accordingly. Maintaining synchronization between the software implementation and supporting documentation will significantly reduce onboarding time for future teams and improve long-term maintainability of the D.A.P. system.

A Recommendations Document was created by our capstone team to give suggestions for the next team to carry on this project. This document will be provided along with other deliverables in documentation folder.

4 Mission Objective and System Requirements Validation

4.1 System-Level Test Overview

The purpose of the D.A.P. System Test was to evaluate the complete Damage Assessment Protocol system and verify compliance with system requirements SR-1 through SR-4. The test was conducted using the Phase B System Test Matrix, which consisted of 26 previously recorded Seaglider mission datasets collected from the Colvos, Edmonds, and Motive missions.

For each test case, pilots executed the D.A.P. system using only the mission `.log` and `.nc` files associated with a particular dive. The `.log` files contained engineering data such as vehicle position, velocity, attitude, and actuator information, while the `.nc` files contained science and environmental measurements including temperature, salinity, and density.

Using the Pilot Guidebook, pilots loaded the mission files into the D.A.P. system and executed the full Digital Twin and Diagnoser pipeline. Upon completion of each run, pilots recorded the Final Damage Assessment Report, including the Primary Diagnosis and computational runtime.

The expected fault condition for each dive was known prior to testing. Therefore, the reported diagnosis produced by the D.A.P. system could be directly compared against the expected fault condition to evaluate overall system performance. The resulting test data were used to verify compliance with SR-1 through SR-4 and determine whether the D.A.P. system achieved the project mission objective.

4.2 SR-1 Verification

Requirement:

The system shall only utilize onboard Seaglider sensor data, including vehicle position, velocity, attitude, and environmental measurements.

Verification Method:

Pilots executed the D.A.P. system using only mission `.log` and `.nc` files obtained from Seaglider deployments. No external measurements, pilot annotations, manually entered parameters, or off-board information were provided to the system during testing.

The 26 system test dives included datasets from the Colvos, Edmonds, and Motive missions. For each dive, the Digital Twin, Mission Comparator, and Diagnoser operated exclusively using data extracted from the onboard mission files.

Results:

All recorded diagnoses were generated using only the mission `.log` and `.nc` files provided to the D.A.P. system.

Conclusion:

SR-1 is verified and passes. The D.A.P. system successfully utilized only onboard Seaglider mission data throughout the entire analysis process.

4.3 SR-2 Verification

Requirement:

The system shall diagnose single wing and rudder loss physical faults within five data transmissions of fault occurrence.

Verification Method:

Pilots used the Pilot Guidebook to execute the D.A.P. system and record the Primary Diagnosis reported in the Final Damage Assessment Report for each dive.

Three fault-case groups were evaluated:

- Five Edmonds dives containing 50
- Five Edmonds dives containing 50
- Five Motive dives containing rudder-loss faults.

For each dive, the system-generated diagnosis was recorded and compared against the known fault condition.

Results:

For the Edmonds rudder-loss dives, the system identified that a fault existed in three of the five datasets; however, the reported fault type was incorrect.

For the Edmonds wing-loss dives, the system identified two of the five datasets as fault cases and partially matched the expected wing-loss condition by reporting a 25

For the Motive rudder-loss dives, the system identified a fault in four of the five datasets and correctly reported a 50

Conclusion:

The requirement as written only requires the system to diagnose a wing-loss or rudder-loss fault within five data transmissions. It does not explicitly require the diagnosis to correctly identify the fault type or severity.

Because the system produced wing-loss or rudder-loss diagnoses within the required transmission window, SR-2 is considered a pass.

However, the project team recommends revising this requirement in future development efforts to include:

- Correct identification of the fault type.
- Correct identification of nominal cases.
- Acceptable fault-severity tolerance ranges.
- Agreement between diagnosed faults and actual field conditions.

4.4 SR-3 Verification

Requirement:

The system shall detect and isolate single wing and rudder loss physical faults in a deployed Seaglider with at least 60

Verification Method:

The confidence metric was derived from the Digital Twin and Diagnoser subsystems.

The Digital Twin confidence metric was defined as:

[Confidence= Points Within Allowable Margins$_{\overline{\text{Total Dive Data Points}\times 100]}}$

The Diagnoser compares mission data against Digital Twin outputs and verifies whether comparison metrics remain within established allowable error margins. Because all Diagnoser classifications rely on Digital Twin simulation outputs, overall system confidence is limited by Digital Twin accuracy.

Results:

Digital Twin validation testing produced confidence values below the required 60

Representative results included:

- 50

- 50
- 50

The Digital Twin failed to satisfy its allowable error margins, preventing the system from achieving the required confidence threshold.

Although the Diagnoser correctly processed the Digital Twin outputs and generated fault classifications, the overall confidence metric remained below 60

Conclusion:

SR-3 fails.

The Digital Twin did not achieve the required confidence level and therefore the overall D.A.P. system could not satisfy the 60

The project team additionally recommends that future versions of this requirement include confidence verification for nominal cases to ensure the system can identify both nominal and faulted vehicle behavior with sufficient confidence.

4.5 SR-4 Verification

Requirement:

The system must not exceed one hour of computational runtime for a single dive data set analysis.

Verification Method:

Pilots recorded the total runtime reported by the D.A.P. system for each of the 26 system test dives. Runtime values were recorded in seconds and converted to hours for comparison against the requirement threshold.

Results:

All recorded runtimes were below the one-hour limit.

The maximum observed runtime was approximately 1946 seconds (0.54 hours), which remained below the one-hour requirement threshold.

Conclusion:

SR-4 is verified and passes.

The D.A.P. system successfully analyzed every test case within the required one-hour runtime limit.

The project team notes that this requirement may not remain satisfied if additional fault cases are incorporated into the fault library. Computational runtime increases as additional fault matrices are evaluated. Future versions of this requirement should consider limiting runtime relative to dive duration rather than using a fixed one-hour threshold.

4.6 Mission Objective Validation

Mission Objective:

Develop a tool that uses historical Seaglider datasets to detect and isolate single wing-loss or rudder-loss faults within five datasets of occurrence in a deployed Seaglider with 60

Results:

- Non-intrusive operation requirement satisfied.
- Historical mission datasets successfully used as the sole system input.
- Fault diagnoses were generated during testing.
- Required 60
- Fault isolation performance did not consistently match expected fault conditions.

Conclusion:

The D.A.P. system successfully demonstrated non-intrusive fault assessment using historical Seaglider mission data. However, the system did not achieve the required 60 percent confidence threshold and did not consistently isolate the correct fault condition across all test cases.

Therefore, the D.A.P. system did not fully satisfy the mission objective.

5 Test Plans and Procedures

This section provides an overview of the test plans and procedures created throughout development of the Damage Assessment Protocol (D.A.P.) system. The purpose of this section is to guide future teams to the correct testing documents and explain what each test was used for. Detailed procedures, test matrices, and execution steps should be referenced directly from the files listed in each subsection.

5.1 Data Collection Tests

Data collection tests were used to collect physical, experimental, and mission-based data for the Digital Twin, Dynamics and Forces model, and Diagnoser.

5.1.1 Wind Tunnel Testing

Wind Tunnel Testing was used to collect force and moment data for Seaglider model configurations. The Wind Tunnel folder contains the following files:

- CAD folder
- `alldata.csv`
- `DR.3x3.config.plots.named.figures.m`
- `Wind.Tunnel.Test.Procedure.pdf`
- `WT Run Log 21-22 May - Sheet1.pdf`
- `WT Run Log 21-22 May.xlsx`

Future teams should reference `Wind.Tunnel.Test.Procedure.pdf` to understand the wind tunnel setup, test environment, equipment, angle of attack sweeps, sideslip sweeps, nominal configuration, and off-nominal damage modes. The CAD folder should be used to identify the physical wind tunnel model geometry used during testing. The run log files and `alldata.csv` should be used to review the collected wind tunnel data, while `DR.3x3.config.plots.named.figures.m` should be used to understand or reproduce the post-processing of the wind tunnel data.

5.1.2 Tank Testing

Tank Testing was used to observe Seaglider motion under controlled water-tank conditions and evaluate how wing and rudder damage affected vehicle behavior. The Tank Test folder contains the following file:

- `Tank.Test.Procedure.pdf`

Future teams should reference `Tank.Test.Procedure.pdf` to understand the test motivation, test environment, safety considerations, equipment, Seaglider control limits, nominal and off-nominal damage modes, pitch sweep, roll sweep, VBD sweep, expected results, acceptance criteria, and test execution steps.

5.1.3 Field Deployment Testing

Field Deployment Testing was used to collect real mission data from nominal and faulted Seaglider dives. The Field Test folder contains the following file:

- `Field.Test.Procedure.pdf`

Future teams should reference `Field.Test.Procedure.pdf` to understand the field test motivation, test environment, safety considerations, required equipment, Seaglider control limits, nominal and off-nominal fault cases, requested dive profiles, expected results, acceptance criteria, and field execution procedure.

5.2 Subsystem Tests

Subsystem tests were used to verify individual components of the D.A.P. system before full system validation.

5.2.1 Dynamics Testing

Dynamics Testing was used to verify and validate the Dynamics model by comparing simulated outputs to real nominal mission data. The subsystem test folder contains the following Dynamics files:

- `Dynamics.Test.Plan.pdf`
- `Dynamics.Test.Procedure.pdf`

Future teams should reference `Dynamics.Test.Plan.pdf` to understand the test objective, requirements being verified, parameters, variables, measurements, test needs matrix, and test matrix. Future teams should reference `Dynamics.Test.Procedure.pdf` for the step-by-step process used to run the Dynamics validation test, including setup, software initialization, state vector assembly, control vector assembly, integration loop, post-processing, evaluation, and shutdown.

5.2.2 Forces Testing

Forces Testing was used to verify that the Seaglider forces code correctly calculates lift and drag from CFD coefficient outputs and compares them against the Hydrodynamic Model. The subsystem test folder contains the following Forces file:

- `Forces.Test.Procedure.pdf`

Future teams should reference `Forces.Test.Procedure.pdf` to understand the test objective, required measurements, equipment, and procedure used to validate lift and drag force calculations.

5.2.3 Digital Twin Testing

Digital Twin Testing was used to verify that the Digital Twin could take in dive files, extract required parameters, and output simulated data for nominal and off-nominal Seaglider configurations. The subsystem test folder contains the following Digital Twin file:

- `Digital.Twin.Test.Procedure.pdf`

Future teams should reference `Digital.Twin.Test.Procedure.pdf` to understand the Digital Twin test objective, equipment required, scope, test setup, mission dive configurations, test procedure, anomalies, shutdown process, and sign-off requirements.

5.2.4 Fault Detector Testing

Fault Detector Testing was used to verify the mission comparator/fault detector logic used by the Diagnoser. The subsystem test folder contains the following Fault Detector file:

- `Fault Detector Test Procedure.pdf`

Future teams should reference `Fault Detector Test Procedure.pdf` to understand how the Fault Detector verifies trimming of the first and last 20 percent of dive profiles, percent difference calculations, window-based scoring, sustained deviation detection, and failure output generation.

5.2.5 Fault Isolator Testing

Fault Isolator Testing was used to verify that the Fault Isolator could identify the lowest percent difference between mission data and Digital Twin fault cases. The subsystem test folder contains the following Fault Isolator files:

- `Fault.Isolator.Test.Plan-1.pdf`
- `Fault.Isolator.Test.Procedure-1.pdf`

Future teams should reference `Fault.Isolator.Test.Plan-1.pdf` to understand the test objective, parameters, variables, test matrix, and traceability for Fault Isolator verification. Future teams should reference `Fault.Isolator.Test.Procedure-1.pdf` for the MATLAB setup, data ingestion, comparator execution, fault isolator ranking, and results verification process.

5.2.6 Diagnoser Testing

Diagnoser Testing was used to verify the full Diagnoser function, including mission data input, comparison against fault cases, fault isolation, ranked table generation, and best-fit fault case output. The subsystem test folder contains the following Diagnoser files:

- `Diagnoser.Test.Plan.pdf`
- `Diagnoser.Test.Procedure.pdf`

Future teams should reference `Diagnoser.Test.Plan.pdf` to understand the Diagnoser test objective, scope, parameters, variables, test matrix, and traceability. Future teams should reference `Diagnoser.Test.Procedure.pdf` to understand the step-by-step validation process, including environment initialization, mission data loading, Digital Twin library loading, comparator execution, fault isolator ranking, and results verification.

5.2.7 Pilot Guidebook Test Plan and Procedure

The Pilot Guidebook Test Plan and Pilot Guidebook Test Procedure were developed to verify that the Pilot Guidebook satisfies the documentation subsystem requirements and can successfully guide a pilot through operation of the D.A.P. system.

Future teams should refer to the Pilot Guidebook Test Plan to understand the validation objectives, test parameters, variables, test matrix, and requirement traceability used during documentation verification. The test plan identifies the mission datasets used during validation and the expected diagnoses associated with each dataset.

Future teams should refer to the Pilot Guidebook Test Procedure for the step-by-step validation workflow used during testing. The procedure outlines how pilots use the Pilot Guidebook to execute the D.A.P. system, generate a Final Damage Assessment Report, interpret D.A.P. outputs, and understand how the final diagnosis is generated.

When modifications are made to the Pilot Guidebook, future teams should review and update both the Pilot Guidebook Test Plan and Pilot Guidebook Test Procedure to ensure the documentation continues to satisfy subsystem requirements and remains usable by Seaglider pilots.

5.3 System Tests

System tests were conducted to evaluate the fully integrated D.A.P. system after subsystem-level verification was completed. These tests verified that the Digital Twin, Fault Detector, Fault Isolator, Diagnoser, and Documentation subsystems could operate together to produce a final fault diagnosis using real Seaglider mission data.

5.3.1 D.A.P. System Validation

The D.A.P. System Validation Test was developed to evaluate the complete Damage Assessment Protocol pipeline using previously collected Seaglider mission datasets. This testing effort focused on system-level verification of requirements SR-1, SR-2, and SR-4 and served as the final validation activity for the integrated system.

Future teams should reference the following documents:

- D.A.P. System Test Plan – Phase B
- D.A.P. System Test Procedure

The D.A.P. System Test Plan – Phase B defines the overall system-level test campaign, including the test objective, requirements being verified, test variables, measurements, test needs, and the 26-run test matrix used throughout validation. The plan explains how historical mission datasets from the Colvos, Edmonds, and Motive deployments were used to evaluate the performance of the complete D.A.P. system.

The D.A.P. System Test Procedure provides the detailed execution steps used during system testing. The procedure describes the required software environment, MATLAB version requirements, required D.A.P. system files, mission data folders, setup process, and execution workflow used by the pilots. It also documents how each test case was processed using mission `.log` and `.nc` files, how fault deviation gains were selected, and how Primary Diagnosis and Runtime were recorded for each test case.

Together, these documents provide the complete methodology used to evaluate the integrated D.A.P. system and should be referenced whenever future teams perform system-level validation, regression testing, or verification of updated D.A.P. system.

6 Suggestions for Future Development

Throughout the development of the Damage Assessment Protocol (D.A.P.) system, several areas were identified where future work could significantly improve system accuracy, robustness, usability, and overall mission effectiveness. Rather than duplicating all recommendations within this continuity document, the project team created a dedicated *Recommendations for Continuation of the D.A.P. System* document to serve as a centralized reference for future development efforts.

The recommendations document contains subsystem-specific guidance developed by the 2026 ROFASO Capstone Team based on design experiences, testing observations, validation results, and known system limitations encountered throughout development.

Future teams should review this document before beginning any major modifications to the D.A.P. system. Particular attention should be given to the following topics:

- Digital Twin architecture recommendations and lessons learned.

- Guidance for implementing an independent Digital Twin controller.
- Recommendations for learning and utilizing Seaglider mission data sources.
- Dynamics model improvements and modeling assumptions.
- Updates to vehicle mass properties and trim parameters.
- Numerical integration and simulation stability improvements.
- Added mass and inertia matrix enhancements.
- Artificial damping implementation and system identification approaches.
- Improvements to force and moment modeling methodologies.
- Computational Fluid Dynamics (CFD) recommendations.
- Physical testing recommendations, including wind tunnel and water tunnel testing.
- Ocean current modeling integration opportunities.
- Diagnoser improvements, including dive grouping analysis.
- Diagnoser tolerance training using larger mission datasets.

The recommendations document should be considered a companion document to this continuity report. While this continuity document focuses on preserving the current system design, implementation, and testing history, the recommendations document focuses on future development opportunities and lessons learned by the project team. Future teams are strongly encouraged to review both documents together before initiating new development efforts.

7 Conclusion

This continuity document was created to provide future capstone teams with a complete overview of the Damage Assessment Protocol (D.A.P.) system and the work completed during the 2025–2026 project cycle. The document serves as a roadmap to the major deliverables, supporting documentation, design references, testing artifacts, and recommendations generated throughout development.

The D.A.P. system consists of three primary subsystems: the Digital Twin, Diagnoser, and Documentation subsystem. The Digital Twin simulates nominal and faulted Seaglider behavior using vehicle dynamics, force models, and mission data. The Diagnoser compares mission data against Digital Twin outputs to identify the most likely fault case. The Documentation subsystem includes the Pilot Guidebook and supporting validation documents that allow pilots to operate the D.A.P. system and interpret its outputs.

Future teams should begin by reviewing the Digital Twin documentation, particularly the Dynamics and Forces documents and Mass and Inertia Matrix documentation, to understand the underlying vehicle model used throughout the project. Teams should then review the Diagnoser design documentation and associated validation tests to understand how mission data is compared against Digital Twin outputs to generate fault classifications.

For documentation-related work, future teams should review the Pilot Guidebook, Pilot Guidebook Test Plan, and Pilot Guidebook Test Procedure before making modifications to the user workflow or output interpretation process. Any future updates to the Pilot Guidebook should be accompanied by updates to the corresponding validation documentation.

Testing artifacts contained within this project include wind tunnel testing, tank testing, field testing, subsystem validation testing, and integrated system validation testing. Future teams should

review these documents before conducting additional testing to ensure consistency with previous verification methods and data collection practices.

Finally, future teams should review the Suggestions Document before beginning development. This document summarizes lessons learned throughout the project and provides recommendations regarding Digital Twin improvements, Diagnoser enhancements, documentation revisions, requirement modifications, and future testing efforts. The Suggestions Document should be considered the primary roadmap for future development activities.

Although the project successfully developed and integrated the Digital Twin, Diagnoser, and Documentation subsystems, the mission objective was not fully achieved due to limitations in Digital Twin accuracy and the resulting inability to satisfy the required confidence threshold. The work completed during this project establishes a functional foundation for future teams and provides the documentation, testing infrastructure, and recommendations necessary to continue development of the Damage Assessment Protocol system.